

Project Phase 3

Data Systems – Spring 2026

Due Date: 23:59 Hrs, 20th April, 2026

1 Instructions

- Teams remain the same as in the previous phases.
- This phase builds upon the codebase created in Phases 1 and 2.
- All commits for this phase are to be done in the `main` branch of the repository, which is added to your team on GitHub Classroom. You may create separate branches, but you are supposed to merge them into the main branch by the due date.
- No personal queries will be entertained. All queries regarding the project must be posted in the doubts document. **Only doubts clarified in the doubts document will be considered for grading.**
- The late days form must be filled on the day of submission if any commit is made past the due date in the main branch. You must provide the correct number of late days taken.
- Plagiarism checks will be conducted on all submissions. Copied code, whether partially or fully, will be heavily penalized.
- Not following any of the instructions will result in a penalty.

Important Constraints and Implementation Notes

- The deliverables in this phase take tables as input.
- The overall program flow must not be modified. The reference flow can be found in `docs/flow.png`.
- The `cursor` and `bufferManager` classes must be used for handling reading and writing to the pages.
- Private variables of any class must not be changed to public. Access should be provided only through appropriate getter and setter methods.
- The values of `BLOCK_SIZE` and `BLOCK_COUNT` must remain unchanged.
- The implemented commands should run in a reasonable amount of time.
- Any additional changes or extensions must not affect the correctness or behavior of already implemented table/graph queries.

2 Concurrency Control Implementation

Implement any **one** of the following concurrency control techniques from Chapter 21 of the Elmasri textbook:

- **Section 21.1: Two-Phase Locking Techniques for Concurrency Control**
 - Strict Two-Phase Locking must be implemented.
 - The wait-die deadlock prevention protocol is to be implemented. Deadlock detection mechanisms do not need to be implemented.
- **Section 21.2: Concurrency Control Based on Timestamp Ordering**
 - The Strict Timestamp Ordering algorithm must be implemented.
 - **Aborted transactions are assumed to restart at a later time with new timestamps.**
- **Only the specified protocols are to be implemented. Any other protocol implementations will not be considered.**

You must implement either a `LockManager` or a `TimestampManager` within your existing `SimpleRA` codebase.

Required Scope

- The implementation must operate at **page granularity**.
- `LockManager`/`TimestampManager` should maintain concurrency metadata for each page in memory:
 - For locking: lock type and owning transaction(s).
 - For timestamp ordering: read timestamp and write timestamp.
- Concurrency control must be enforced **before any page is read or written**.
- Existing page access flow using `cursor` and `bufferManager` must be preserved.

Input Format

TRANSACTION <input_file>

Example: TRANSACTION schedule1.txt

The input text file will contain a transaction schedule, with each line representing one operation. The input files will be located in the `/data` directory. The corresponding output files must also be stored in the `/data` directory, following the naming convention:

<input_filename>_output.txt

For example, if the input file is named `test1.txt`, the output file should be saved as `test1_output.txt` in the same `/data` directory.

Supported commands in the input file:

```
BEGIN T1
READ T1 table_name page_number
```

```
WRITE T1 table_name page_number
COMMIT T1
```

Assume that the first transaction begins at $t=1$ and each step (begin, commit, read, write) takes 1 unit of time.

Example input file:

```
BEGIN T1
BEGIN T2
BEGIN T3
READ T1 student 3
READ T2 student 3
WRITE T1 student 3
READ T3 student 3
WRITE T2 student 3
WRITE T3 student 3
COMMIT T1
COMMIT T2
COMMIT T3
```

Expected Behaviour

- Transactions must be processed in the given schedule order.
- If a conflict occurs, process it according to the algorithm chosen.

Implementation Restrictions

- The implementation must only be a **simulation of concurrency control**.
- Actual modification of page contents is **not required**. The focus of this phase is on correct concurrency control decisions and execution logging.
- **Threading, multiprocessing, asynchronous execution, or any other form of parallel execution is not required to be used.**
- The output must clearly show whether each operation is allowed, rejected, aborted, restarted, or committed.

Expected Output

The output file must contain the execution log produced by the implemented concurrency control protocol.

Example output for strict two-phase locking using the wait-die deadlock prevention mechanism:

```
BEGIN T1
BEGIN T2
BEGIN T3

T1 requests shared lock(student 3)
Lock granted
```

READ T1 student 3

T2 requests shared lock(student 3)

Lock granted

READ T2 student 3

T1 requests exclusive lock(student 3)

T1 waits (T2 holds shared lock, T1 is older so allowed to wait)

T3 requests shared lock(student 3)

Lock granted

READ T3 student 3

T2 requests exclusive lock(student 3)

T2 dies (younger than T1, cannot wait)

ABORT T2

ROLLBACK T2

unlock(student 3) by T2

T1 still waits because T3 holds shared lock

T3 requests exclusive lock(student 3)

T3 dies (younger than T1, cannot wait)

ABORT T3

ROLLBACK T3

unlock(student 3) by T3

T1 exclusive lock granted

WRITE T1 student 3

COMMIT T1

unlock(student 3) by T1

Restart T2

T2 requests shared lock(student 3)

Lock granted

READ T2 student 3

T2 requests exclusive lock(student 3)

Lock granted

WRITE T2 student 3

COMMIT T2

unlock(student 3) by T2

Restart T3

T3 requests shared lock(student 3)

```

Lock granted
READ T3 student 3
T3 requests exclusive lock(student 3)
Lock granted
WRITE T3 student 3
COMMIT T3
unlock(student 3) by T3

```

Example output for timestamp ordering using the strict timestamp ordering algorithm:

```

BEGIN T1 [TS = 1]
BEGIN T2 [TS = 2]
BEGIN T3 [TS = 3]

T1 READ student page 3 -> allowed [read_TS(student page 3) = 1]
T2 READ student page 3 -> allowed [read_TS(student page 3) = 2]
T1 WRITE student page 3 -> rejected [read_TS(student page 3) = 2 but TS(T1) = 1]
ABORT T1
ROLLBACK T1

T3 READ student page 3 -> allowed [read_TS(student page 3) = 3]
T2 WRITE student page 3 -> rejected [read_TS(student page 3) = 3 but TS(T2) = 2]
ABORT T2
ROLLBACK T2

T3 WRITE student page 3 -> allowed [write_TS(student page 3) = 3]
T3 commits [write_TS(page 3) = 3, read_TS(student page 3) = 3]

RESTART T1 as T1' [TS = 11]
T1' READ student page 3 -> allowed [read_TS(student page 3) = 11]
T1' WRITE student page 3 -> allowed [write_TS(student page 3) = 11]
T1' commits [write_TS(page 3) = 11, read_TS(student page 3) = 11]

RESTART T2 as T2' [TS = 15]
T2' READ student page 3 -> allowed [read_TS(student page 3) = 15]
T2' WRITE student page 3 -> allowed [write_TS(student page 3) = 15]
T2' commits [write_TS(page 3) = 15, read_TS(student page 3) = 15]

```

The output formats above are just examples, you are allowed to use any format as long as the output file contains all the required information.

Error Handling

- If the specified table does not exist, an error must be thrown - **SEMANTIC ERROR: Relation doesn't exist**
- If the specified page number does not exist in the table, an error must be thrown -

SEMANTIC ERROR: Page doesn't exist

3 Report

Submit a `Report3.md` in the `docs` directory including:

- Detailed explanation of the implementation.
- Reasoning for choosing one concurrency control technique over the other.
- Assumptions (under a separate **Assumptions** heading).
- Contribution of each team member.

4 Evaluation

An in-person evaluation and viva will be held to assess this phase.